

notify_stock_price.py - 코드만

설명(주식)을 걷어내고 코드 줄만 남긴 것입니다. 구조를 한눈에 보라고 만든 자료이고,
같은 폴더의 notify_stock_price.py 이 원본입니다 - 주식은 그 파일에 그대로 있습니다.

한 줄이 길어 넘어간 곳은 다음 줄에 안쪽으로 들여 이어 붙였습니다.

이모지(☞ ☞ ☞ ☞ ...)는 이 PDF 글꼴에 없어 네모로 보입니다 - 원본에는 정상입니다.

※ 굵은 글씨 = 이번 주에 새로 생긴 줄입니다 (46줄).

bigd_02_telegram 폴더의 파일들에 없던 줄만 굵게 했습니다 - 지난주 코드를 다른 파일로
 옮겨 온 줄은 새 코드가 아니므로 보통 글씨입니다.

=====

```
import os
import time
import requests
from dotenv import load_dotenv

STOCK_CODE = "005930"

load_dotenv()

TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")

def fetch_naver_current_price(code: str, retries: int = 2) -> dict:
    url = f"https://m.stock.naver.com/api/stock/{code}/basic"
    for attempt in range(1, retries + 1):
        try:
            response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"},
                                    timeout=3)
            if response.status_code != 200:
                print(f"☞ [{code}] 네이버 현재가 조회 실패 (응답 코드: {response.status_code},
                    {attempt}/{retries}번째 시도)")
            else:
                data = response.json()
                price_str = data.get("closePrice", "0").replace(",","")
                price = int(price_str) if price_str.isdigit() else 0
                if price <= 0:
                    print(f"☞ [{code}] 가격을 숫자로 읽지 못했습니다 "
                        f"(closePrice: {data.get('closePrice')}!r},
                          {attempt}/{retries}번째 시도)")
                else:
                    return {
                        "code": code,
                        "name": data.get("stockName", "알 수 없음"),
                        "price": price,
                        "rate": float(data.get("fluctuationsRatio", "0") or "0"),
                        "is_open": data.get("marketStatus") == "OPEN",
                        "traded_at": data.get("localTradedAt"),
                    }
        except Exception as e:
            print(f"☞ [{code}] 네이버 현재가 조회 중 오류 발생: {e} ({attempt}/{retries}번째 시도)")

    if attempt < retries:
        time.sleep(1)

    return None
```

```

def send_telegram_message(text: str) -> bool:
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    data = {"chat_id": TELEGRAM_CHAT_ID, "text": text, "parse_mode": "HTML"}
    try:
        response = requests.post(url, data=data, timeout=15)
        return response.status_code == 200
    except Exception as e:
        print(f"🚫 텔레그램 메시지 전송 중 오류 발생: {e}")
        return False


def format_rate_badge(price: int, rate: float) -> str:
    prefix = f"{price:,}원"
    if rate > 0:
        return f"{prefix} 📈 +{rate}%"
    if rate < 0:
        return f"{prefix} 📉 {rate}%"
    return f"{prefix} 📊 0.0%"


info = fetch_naver_current_price(STOCK_CODE)

if info is None:
    print("🚫 현재가를 가져오지 못했습니다. 네이버 API 상태를 확인해 주세요.")
else:
    telegram_message = (
        f"📊 {info['name']} ({STOCK_CODE})"
        f"\n    {format_rate_badge(info['price'], info['rate'])}"
    )

    if send_telegram_message(telegram_message):
        print("📧 현재가 메시지를 텔레그램으로 전송했습니다!")
    else:
        print("🚫 현재가 메시지 전송에 실패했습니다.")

```